

ASSIGNMENT BRIEF

HTU Course No: 10203420	HTU Course Name: Secure System Design & Development
BTEC Unit Code:	BTEC UNIT Name:



Student Name/ID Number/Section	
HTU Course Number and Title	10203420 Secure System Design & Development
BTEC Unit Code and Title	
Academic Year	2024-2025 2
Assignment Author	Fawaz Khasawneh
Course Tutor	Fawaz Khasawneh - hamza alkofohi
Assignment Title	MalAnalysis
Assignment Ref No	1
Issue Date	07/05/2025
Formative Assessment dates	From 08/05/2025 to 29/05/2025
Submission Date	16/06/2025
IV Name & Date	Sultan Alrushdan 06/05/2025
Submission Format	
Each student is expected to individually submit his/her work , including: 1. Report explaining all the findings of your analysis including screenshots. 2. Answers to all questions. 3. Student Declaration Form. Report guidelines: The report should be written in a concise, formal business style using single spacing and font size 12 with the use of headings, paragraphs, and subsections as appropriate (a cover page, table of contents, and an introduction to provide an overview of your report.). The expected word limit is about 5000 words, although you will not be penalized for exceeding the total word limit. The report must be supported with research and referenced using the Harvard referencing system. Caution DO NOT execute the sample on your host machine. Only use an isolated virtual machine. Notes: • If you do not see the Turnitin report when you initially submit your report, contact your instructor immediately. • Resubmission: If you lose more than 4 Ps, you will NOT be eligible for resubmission. Soft copy submissions should be done through the university's eLearning system https://elearning.htu.edu.jo by the deadline assigned above.	
Unit Learning Outcomes	

LO1 Analyze the Fundamentals of Malware and its Impact on Systems

LO2 Apply Static and Dynamic Analysis Techniques to Investigate Malware Behavior

LO3 Reverse Engineer and Debug Malware to Extract Hidden Functionality

LO4 Conduct an End-to-End Malware Analysis and Report Findings Professionally

Assignment Brief and Guidance

A suspicious executable named BackdoorBeacon.exe was discovered on a compromised internal server of a company, **IntraTech Systems**. Network analysts observed unusual outbound traffic to a non-standard port (TCP 4444) during off-hours. Initial antivirus scans returned no hits. Your task is to **analyze this executable using static and dynamic methods** to understand its purpose, behavior, persistence mechanisms, and any hidden or advanced functionalities. You must also produce a **professional malware analysis report**.

Based on your analysis, answer the following questions and add screenshots whenever needed

1. What type of malware is BackdoorBeacon.exe, and what behaviors support your classification?
2. How did you configure your virtual environment to safely analyze this sample, and which tools did you use?
3. What were the key findings from your static analysis (such as, strings, PE sections, hashes, imports, etc.)?
4. What did runtime monitoring (using tools such as, Process Monitor, Regshot, Wireshark, etc.) reveal about the malware's behavior?
5. What persistence mechanisms does the malware employ (such as, registry keys, mutexes, services)?
6. What network communication does the malware attempt, and what does it reveal about potential command-and-control activity?
7. What notable functions, control flows, or routines did you identify through disassembly with IDA free or Ghidra?
8. When debugging the sample, what key behavior did you uncover by tracing stack values, registers, or memory changes, you should choose at least one function to analyze and track set of parameter(s) it is taking, stack update, saved registers, memory changes, etc.
9. Were there any signs of obfuscation or packing in the malware? If so, how did you unpack or bypass them?
10. How does the malware evade analysis or detect debugging environments?
11. Does the sample perform any injection, process hollowing, or other advanced execution techniques? Provide evidence.
12. Were any kernel-level behaviors observed (such as, SSDT hooking, driver loading, IDT manipulation, etc.)?
13. Did the malware use native API calls (e.g., from ntdll.dll) instead of standard Windows API functions? What advantage does this provide?
14. What are the most critical Indicators of Compromise (IoCs) extracted from your analysis?
15. Summarize the overall functionality and goals of the malware. What systems or data does it target?
16. Based on your findings, what containment and mitigation steps would you recommend?

Learning Outcomes and Assessment Criteria			
Learning Outcome	Pass	Merit	Distinction
LO1 Analyze the Fundamentals of Malware and its Impact on Systems	P1 Understand Malware Analysis Concepts. Define different malware types, goals of malware analysis, and the importance of incident response. P2 Set Up a Virtual Machine for Malware Analysis. Configure a safe environment for malware analysis using VMware, snapshots, and networking setups. P3 Perform Basic Static Analysis. Extract useful information from malware samples without execution using tools like strings, PE analysis, and hashing. P4 Execute Basic Dynamic Analysis. Monitor malware behavior in a controlled environment using Process Monitor, Process Explorer, Regshot, and Wireshark. P5 Analyze Windows Internals and APIs. Examine Windows system components, processes, threads, mutexes, and services to understand how malware interacts with the OS.		

LO2 Apply Static and Dynamic Analysis Techniques to Investigate Malware Behavior	P6 Examine x86 Assembly and Reverse Engineering Basics. Identify assembly instructions, register usage, memory addressing, function calls, and stack operations. P7 Disassemble Malware Using IDA Pro or Ghidra. Recognize code constructs, analyze functions, and identify important cross-references within assembly code. P8 Debug Malware in User Mode Using OllyDbg. Step through malware execution, set breakpoints, modify memory, and analyze function calls using OllyDbg.	M1 Debug and Analyze Malware Behavior deeply. Use OllyDbg and WinDbg to analyze malware execution flow, detect anti-debugging techniques, and modify execution. M2 Reverse Engineer Malware Code Using IDA Pro. Deconstruct malware logic by analyzing disassembled code, identifying obfuscation methods, and reconstructing hidden functionalities.	
LO3 Reverse Engineer and Debug Malware to Extract Hidden Functionality		M3 Investigate Advanced Malware Techniques. Identify malware persistence techniques, including registry modifications, DLL injection, and process hollowing.	D1 Perform your own research on how to bypass Malware Anti-Reversing and Obfuscation Techniques. Identify and defeat anti-debugging, code packing, and polymorphic malware techniques. D2 Perform your own research on the techniques used in reverse engineering for the Kernel-Mode Malware, debugging rootkits, analyzing kernel drivers, and detecting privilege escalation.

LO4 Conduct an End-to-End Malware Analysis and Report Findings Professionally			D3 Conduct a Full Malware Analysis and Generate a Report. Perform a complete end-to-end malware investigation, including behavioral analysis, reverse engineering, and mitigation recommendations.